Below are **three (Apache2 script, Java script, Python script) production-ready scripts** you can use to send **H2H (Host-to-Host) SWIFT MX (ISO20022) pacs.008** messages over **raw TLS sockets** to our host `devmindgroup.com:443`

Each script:

- Establishes a **TLS socket** with **SNI / hostname verification**.
- Validates **TCP/TLS reachability** before send.
- Validates the **pacs.008 XML** against the XSD (you must download the official XSD and point each script to it).
- Builds a **length-prefixed FIN-like envelope placeholder** (4-byte big-endian length + custom header) — replace the header with the real FIN envelope required by your bank.
- Sends the framed XML over the TLS socket.
- Waits for a framed response and **parses ACK/NACK** (basic parsing).
- Implements **retry logic (exponential backoff with jitter)** for transient network errors.
- Logs success/failure and saves the server response + metadata to a local file (JSON + raw response).
- Performs required-field checks before sending.

Important (before running):

1. **XSD:** Download the correct `pacs.008` XSD (e.g., `pacs.008.001.08`) and place it where each script expects (see `PACS_XSD_PATH` / `XSD_PATH` variables). Without that, XML validation will fail.
2. **Certificates / MTLS:** If the bank requires mutual TLS, place your client cert / key and set the `CLIENT_CERT` / `CLIENT_KEY` variables.
3. **Envelope:** Replace the simple placeholder envelope with the *exact* envelope required by the counterparty (FIN envelope bytes).
4. **Production controls:** Store keys/certs in a secrets manager (Vault/HSM). Do not commit them to source control.

Our receiver data (used in XML template)

- Bank Name: `DFCU BANK LIMITED`
- Bank Address: `PLOT 26 KYADONDO ROAD, UGANDA`
- SWIFT CODE: `DFCUUGKA`
- Account Name: `SHAMRAYAN ENTERPRISES`
- Account Number: `02650010158937`
- Host/SNI: `devmindgroup.com:443`

Folder structure

```
h2h_sender_project/
├── mx_payload.xml                # Sample pacs.008 message
├── xsd/
│   └── pacs.008.001.08.xsd       # ISO20022 XSD for validation
├── responses/                    # Directory to store ACK/NACK responses
├── h2h_sender_prod.py            # Python raw TLS sender
├── h2h_sender_prod.php           # PHP raw TLS sender
└── H2HSenderProd.java            # Java raw TLS sender
```

Make sure `responses/` exists or scripts will create it automatically.

Sample `pacs.008` XML template (populated & annotated with our bank details)

This is a **pacs.008** (FI to FI Customer Credit Transfer) style — you must adapt it to your bank message rules and ensure the XSD version matches. I include inline comments (XML comments) to annotate required fields.

Save as **`mx_payload.xml`**. **(copy script code below):**

```
<?xml version="1.0" encoding="UTF-8"?>
<!--
  pacs.008 sample (small). REQUIRED fields annotation:
  - MsgId (GrpHdr/MsgId) : unique business message id (idempotency)
  - CreDtTm (GrpHdr/CreDtTm) : creation timestamp
  - NbOfTxs and CtrlSum in GrpHdr
  - InstdAmt (CdtTrfTxInf/Amt/InstdAmt) : amount + Ccy attribute
  - EndToEndId (PmtId/EndToEndId)
  - CdtrAgt/FinInstnId/BIC : beneficiary bank SWIFT/BIC (DFCUUGKA)
  - CdtrAcct/Id/Othr/Id or IBAN : beneficiary account number
  - Cdtr/Nm : beneficiary name
-->
<Document xmlns="urn:iso:std:iso:20022:tech:xsd:pacs.008.001.08">
  <FIToFICstmrCdtTrf>
    <GrpHdr>
      <MsgId>MSG-20251125-0001</MsgId> <!-- REQUIRED -->
      <CreDtTm>2025-11-25T22:00:00Z</CreDtTm> <!-- REQUIRED -->
      <NbOfTxs>1</NbOfTxs>
      <CtrlSum>500.00</CtrlSum>
      <InstgAgt>
        <FinInstnId>
          <BIC>YOURBANKBIC</BIC> <!-- Replace with sender BIC -->
        </FinInstnId>
      </InstgAgt>
    </GrpHdr>

    <CdtTrfTxInf>
```

```
      <PmtId>
        <InstrId>INSTR-0001</InstrId>
        <EndToEndId>REF-0001</EndToEndId> <!-- REQUIRED -->
      </PmtId>
      <Amt>
        <InstdAmt Ccy="USD">500.00</InstdAmt> <!-- REQUIRED -->
      </Amt>
      <CdtrAgt>
        <FinInstnId>
          <BIC>DFCUUGKA</BIC> <!-- Beneficiary SWIFT (REQUIRED) -->
        </FinInstnId>
      </CdtrAgt>
      <Cdtr>
        <Nm>DFCU BANK LIMITED</Nm> <!-- Beneficiary bank name (REQUIRED) -->
        <PstlAdr>
          <AdrLine>PLOT 26 KYADONDO ROAD, UGANDA</AdrLine>
        </PstlAdr>
      </Cdtr>
      <CdtrAcct>
        <Id>
          <Othr>
            <Id>02650010158937</Id> <!-- Beneficiary account number
(REQUIRED) -->
          </Othr>
        </Id>
      </CdtrAcct>
      <UltmtDbtr>
        <Nm>SHAMRAYAN ENTERPRISES</Nm> <!-- Account Name (REQUIRED if not
using Dbtr) -->
      </UltmtDbtr>
      <RmtInf>
        <Ustrd>Payment for invoice 1234</Ustrd>
      </RmtInf>
    </CdtTrfTxInf>
  </FIToFICstmrCdtTrf>
</Document>
```

Replace `YOURBANKBIC` with your actual sending bank's BIC in `GrpHdr/InstgAgt/FinInstnId/BIC` and update amounts/IDs per message.

1) PHP (Apache2) production-grade H2H (Host-to-Host) sender

Place under an Apache document root or run from CLI. Uses `stream_socket_client` + `ssl://` and `DOMDocument::schemaValidate` for XSD validation.

Files & setup

- `h2h_send.php` (script below)
- `mx_payload.xml` (the template above)
- XSD file path: `xsd/pacs.008.001.08.xsd` (download and place)
- `responses/` directory (script saves responses here)

The PHP (Apache2) — production-ready, with:

- TLS handshake (with optional mTLS)
- Retry/backoff with jitter
- XSD validation + required field checks
- ACK/NACK detection in responses
- Response saved with metadata

h2h_send.php (copy script code below):

```
<?php
// h2h_sender_prod.php
// Usage: php h2h_sender_prod.php mx_payload.xml

declare(strict_types=1);

$HOST = 'devmindgroup.com';
$PORT = 443;
$SNI = 'devmindgroup.com';
$XSD_PATH = __DIR__ . '/xsd/pacs.008.001.08.xsd';
$CA_FILE = '/etc/ssl/certs/ca-certificates.crt';
$CLIENT_CERT = null; // '/secure/path/client.pem' if mTLS required
$CLIENT_KEY = null;  // optional
$RESPONSE_DIR = __DIR__ . '/responses';
@mkdir($RESPONSE_DIR, 0750, true);

$MAX_RETRIES = 4;
$TIMEOUT = 15;

// Logging helper
function log_msg(string $level, string $msg) {
    $line = sprintf("[%s] %s: %s\n", gmdate('c'), strtoupper($level), $msg);
    file_put_contents(__DIR__ . '/h2h_sender.log', $line, FILE_APPEND |
LOCK_EX);
```

```
    echo $line;
}

if ($argc < 2) {
    echo "Usage: php {$argv[0]} <mx_payload.xml>\n";
    exit(2);
}

$payloadFile = $argv[1];
if (!file_exists($payloadFile)) {
    log_msg('error', "Payload file not found: $payloadFile");
    exit(1);
}

$xml = file_get_contents($payloadFile);

// 1) Validate XML against XSD
libxml_use_internal_errors(true);
$dom = new DOMDocument();
$dom->preserveWhiteSpace = false;
$dom->loadXML($xml);

if (!file_exists($XSD_PATH)) {
    log_msg('error', "XSD not found at $XSD_PATH");
    exit(2);
}

if (!$dom->schemaValidate($XSD_PATH)) {
    foreach (libxml_get_errors() as $e) {
        log_msg('error', trim($e->message));
    }
    libxml_clear_errors();
    exit(3);
}
log_msg('info', 'XML validated against XSD');

// 2) Required field check
function required_field_check(string $xml): bool {
    $dom = new DOMDocument();
    $dom->loadXML($xml);
    $xpath = new DOMXPath($dom);
    $xpath->registerNamespace('ns',
'urn:iso:std:iso:20022:tech:xsd:pacs.008.001.08');
    $checks = [
        '//ns:MsgId',
        '//ns:InstdAmt',
        '//ns:EndToEndId',
        '//ns:BIC',
        '//ns:Cdtr/ns:Nm',
        '//ns:CdtrAcct//ns:Othr/ns:Id'
    ];
    foreach ($checks as $q) {
        $res = $xpath->query($q);
        if ($res === false || $res->length === 0) return false;
    }
    return true;
}
```

```
if (!required_field_check($xml)) {
    log_msg('error', 'Required field check failed');
    exit(3);
}
log_msg('info', 'Required fields present');

// 3) Build FIN-like frame
function build_frame(string $xml): string {
    $trace = bin2hex(random_bytes(8)); // 16 hex chars
    $envelope = "FIN1" . chr(1) . $trace;
    $body = $envelope . $xml;
    return pack('N', strlen($body)) . $body;
}

$frame = build_frame($xml);

// 4) Send over TLS with retries
for ($attempt = 1; $attempt <= $MAX_RETRIES; $attempt++) {
    log_msg('info', "Attempt $attempt connecting to $HOST:$PORT");

    $context_opts = [
        'ssl' => [
            'verify_peer' => true,
            'verify_peer_name' => true,
            'cafile' => $CA_FILE,
            'peer_name' => $SNI,
            'capture_peer_cert' => true,
            'ciphers' => 'HIGH:!aNULL:!eNULL',
        ]
    ];
    if ($CLIENT_CERT) $context_opts['ssl']['local_cert'] = $CLIENT_CERT;
    if ($CLIENT_KEY) $context_opts['ssl']['local_pk'] = $CLIENT_KEY;

    $ctx = stream_context_create($context_opts);
    $fp = @stream_socket_client("ssl://$HOST:$PORT", $errno, $errstr,
$TIMEOUT, STREAM_CLIENT_CONNECT, $ctx);

    if (!$fp) {
        log_msg('warn', "Connection failed: $errno $errstr");
        $sleep = (2 ** ($attempt - 1)) + (rand(0,1000)/1000);
        log_msg('info', "Retrying in $sleep seconds...");
        sleep($sleep);
        continue;
    }

    stream_set_timeout($fp, $TIMEOUT);
    fwrite($fp, $frame);
    fflush($fp);

    $lenbin = fread($fp, 4);
    if ($lenbin === false || strlen($lenbin) < 4) { fclose($fp); continue; }
    $length = unpack('Nlen', $lenbin)['len'];
    $resp = '';
    while (strlen($resp) < $length) {
        $resp .= fread($fp, $length - strlen($resp));
    }
```

```
    fclose($fp);

    $ts = gmdate('Ymd_His');
    $respFile = "$RESPONSE_DIR/resp_{$ts}_" . bin2hex(random_bytes(4)) .
".xml";
    file_put_contents($respFile, $resp);
    log_msg('info', "Saved response to $respFile");

    $rdom = @simplexml_load_string($resp);
    if ($rdom) {
        $status = $rdom->xpath('//Status') ?: $rdom->xpath('//GrpSts') ?:
$rdom->xpath('//TxSts');
        $statusText = $status ? (string)$status[0] : 'UNKNOWN';
        log_msg('info', "Parsed status: $statusText");
        if (stripos($statusText,'ACCEPT')!==false ||
stripos($statusText,'ACK')!==false) {
            log_msg('info', 'Message accepted');
            exit(0);
        } elseif (stripos($statusText,'REJECT')!==false ||
stripos($statusText,'NACK')!==false) {
            log_msg('warn', 'Business NACK/REJECT: ' . $statusText);
            exit(3);
        }
    } else {
        log_msg('warn', 'Response not XML or parse failed; saved for
inspection');
    }

    $sleep = (2 ** ($attempt - 1)) + (rand(0,1000)/1000);
    log_msg('info', "Retrying in $sleep seconds...");
    sleep($sleep);
}
log_msg('error', 'Exhausted retries');
exit(4);
```

Run PHP sender: (copy code below):

```
php h2h_sender_prod.php /root/mx_payload.xml
```

Note: **`/root/mx_payload.xml`** assumes the file is placed in the root user's home directory (`/root/`) and you're running the command from there.

Or

Run PHP sender: (copy code below):

```
php h2h_sender_prod.php mx_payload.xml
```

Or

Run PHP sender: (copy code below):

```
php h2h_sender_prod.php /full/path/to/mx_payload.xml
```

Note: **`full/path/to/mx_payload.xml`** is just a placeholder showing you can use any absolute or relative path where your XML payload actually resides.

---

2) Python production-grade H2H (Host-to-Host) sender (recommended baseline)

`h2h_sender_prod.py` fully production-ready for sending SWIFT MX (ISO20022) `pacs.008` messages over raw TLS sockets.

This version includes:

- It **automatically validates** the XML against the XSD.
- Checks **required fields**.
- Handles **TLS handshake** (supports optional mTLS).
- Uses **retry/backoff** with logging.
- Saves **responses** + metadata.
- Detects **ACK/NACK** in the response.

Requirements (copy code below):

```
pip install lxml
```

Place files

- `mx_payload.xml` (template)
- XSD at `xsd/pacs.008.001.08.xsd`
- `responses/` directory

h2h_sender.py (copy script code below):

```
#!/usr/bin/env python3
# h2h_sender_prod.py
import socket, ssl, struct, time, os, uuid, logging, json, sys
from random import uniform
from lxml import etree

# CONFIG - set securely in env or config store for production
HOST = 'devmindgroup.com'
PORT = 443
```

```
SNI = 'devmindgroup.com'
CA_BUNDLE = '/etc/ssl/certs/ca-certificates.crt'
CLIENT_CERT = None  # '/secure/path/client.pem' for mTLS (cert+key combined)
or None
XSD_PATH = 'xsd/pacs.008.001.08.xsd'  # MUST exist
RESPONSE_DIR = 'responses'
os.makedirs(RESPONSE_DIR, exist_ok=True)

TIMEOUT = 15
RETRIES = 4
BACKOFF_BASE = 1.0
JITTER = 0.5

logging.basicConfig(level=logging.INFO, format='%(asctime)s [%(levelname)s]
%(message)s')

# ---- XML Validation ----
def validate_xml_against_xsd(xml_bytes):
    if not os.path.exists(XSD_PATH):
        raise FileNotFoundError(f"XSD not found at {XSD_PATH}")
    xml_doc = etree.fromstring(xml_bytes)
    with open(XSD_PATH, 'rb') as f:
        schema_doc = etree.parse(f)
    schema = etree.XMLSchema(schema_doc)
    if not schema.validate(xml_doc):
        err = schema.error_log
        raise ValueError(f"XSD validation failed: {err.last_error}")
    return xml_doc

def required_field_check(xml_doc):
    nsmap = xml_doc.nsmap
    msg = xml_doc.xpath('//ns:MsgId', namespaces={'ns':
xml_doc.nsmap.get(None)})
    amt = xml_doc.xpath('//ns:InstdAmt', namespaces={'ns':
xml_doc.nsmap.get(None)})
    e2e = xml_doc.xpath('//ns:EndToEndId', namespaces={'ns':
xml_doc.nsmap.get(None)})
    bic = xml_doc.xpath('//ns:BIC', namespaces={'ns':
xml_doc.nsmap.get(None)})
    acct = xml_doc.xpath('//ns:CdtrAcct//ns:Othr/ns:Id', namespaces={'ns':
xml_doc.nsmap.get(None)})
    return bool(msg and amt and e2e and bic and acct)

# ---- FIN Frame ----
def build_frame(xml_bytes, trace=None):
    trace_id = (trace or uuid.uuid4().hex)[:16]
    header = b'FIN1' + bytes([1]) + trace_id.encode('ascii')
    body = header + xml_bytes
    return struct.pack('!I', len(body)) + body

# ---- Socket Helpers ----
def recv_exact(sock, n):
    data = b''
    while len(data) < n:
        chunk = sock.recv(n - len(data))
        if not chunk:
            raise ConnectionError("Connection closed")
```

```
        data += chunk
    return data

def recv_frame(ssl_sock, timeout=TIMEOUT):
    ssl_sock.settimeout(timeout)
    raw_len = recv_exact(ssl_sock, 4)
    size = struct.unpack('!I', raw_len)[0]
    body = recv_exact(ssl_sock, size)
    return body

def parse_ack(xml_bytes):
    try:
        root = etree.fromstring(xml_bytes)
        for tag in ['GrpSts','TxSts','Status','RtrSts']:
            el = root.find('.//{%s}%s' % (root.nsmap.get(None), tag))
            if el is not None:
                return tag, el.text
        return root.tag, etree.tostring(root, pretty_print=True).decode('utf-
8')
    except Exception:
        return None, xml_bytes.decode('utf-8', errors='ignore')

def send_once(xml_bytes):
    context = ssl.create_default_context(cafile=CA_BUNDLE)
    if CLIENT_CERT:
        context.load_cert_chain(certfile=CLIENT_CERT)
    context.check_hostname = True
    context.verify_mode = ssl.CERT_REQUIRED
    context.options |= ssl.OP_NO_SSLv2 | ssl.OP_NO_SSLv3

    raw = socket.create_connection((HOST, PORT), timeout=TIMEOUT)
    ssl_sock = context.wrap_socket(raw, server_hostname=SNI)
    logging.info("TLS handshake ok: cipher=%s", ssl_sock.cipher())
    frame = build_frame(xml_bytes)
    ssl_sock.sendall(frame)
    logging.info("Frame sent (%d bytes payload)", len(xml_bytes))
    resp = recv_frame(ssl_sock)
    ssl_sock.close()
    return resp

def save_response(msgid, resp_bytes):
    timestamp = time.strftime('%Y%m%dT%H%M%SZ')
    fname = os.path.join(RESPONSE_DIR, f"resp_{msgid}_{timestamp}.bin")
    with open(fname, 'wb') as f:
        f.write(resp_bytes)
    meta = {
        "msgid": msgid,
        "timestamp": timestamp,
        "file": fname
    }
    with open(os.path.join(RESPONSE_DIR, f"meta_{msgid}_{timestamp}.json"),
'w', encoding='utf-8') as mf:
        json.dump(meta, mf, indent=2)
    return fname

# ---- Main ----
def main():
```

```
    if len(sys.argv) < 2:
        print(f"Usage: {sys.argv[0]} <mx_payload.xml>")
        sys.exit(1)
    payload_file = sys.argv[1]
    if not os.path.exists(payload_file):
        logging.error(f"Payload file not found: {payload_file}")
        sys.exit(1)

    with open(payload_file, 'rb') as f:
        xml_bytes = f.read()

    xml_doc = validate_xml_against_xsd(xml_bytes)
    if not required_field_check(xml_doc):
        logging.error("Required field check failed")
        sys.exit(1)

    msgid_el = xml_doc.xpath('//ns:MsgId', namespaces={'ns':
xml_doc.nsmap.get(None)})
    msgid = msgid_el[0].text if msgid_el else uuid.uuid4().hex

    for attempt in range(1, RETRIES + 1):
        try:
            logging.info("Attempt %d connecting to %s:%d", attempt, HOST,
PORT)
            resp = send_once(xml_bytes)
            fname = save_response(msgid, resp)
            tag, content = parse_ack(resp)
            logging.info("Parsed ack: %s -> %s", tag, content)
            if content and ('ACCEPT' in str(content).upper() or 'ACK' in
str(content).upper()):
                logging.info("Message accepted")
                return
            if content and ('REJECT' in str(content).upper() or 'NACK' in
str(content).upper()):
                logging.warning("Business rejection: %s", content)
                return
            logging.info("Unknown response; check file %s", fname)
            return
        except (socket.timeout, ConnectionError) as e:
            logging.warning("Network error: %s", e)
        except ssl.SSLError as e:
            logging.error("TLS error: %s", e)
            break
        except Exception as e:
            logging.exception("Fatal error: %s", e)
        sleep = BACKOFF_BASE * (2 ** (attempt - 1)) + uniform(0, JITTER)
        logging.info("Retrying in %.2f seconds", sleep)
        time.sleep(sleep)
    logging.error("Exhausted retries")

if __name__ == '__main__':
    main()
```

How to run PYHON (copy code below):

```
python3 /root/h2h_sender_prod.py /root/mx_payload.xml
```

Note: **`/root/mx_payload.xml`** assumes the file is placed in the root user's home directory (`/root/`) and you're running the command from there.

Or

Run PYHON (copy code below):

```
python3 h2h_sender_prod.py mx_payload.xml
```

Or

Run PYHON (copy code below):

```
python3 h2h_sender_prod.py /full/path/to/mx_payload.xml
```

Note: **`full/path/to/mx_payload.xml`** is just a placeholder showing you can use any absolute or relative path where your XML payload actually resides.

---

3) Java (JDK 11+) production-grade H2H (Host-to-Host) sender

`H2HSender.java` — uses `SSLSocket`, validates XML against XSD via `javax.xml.validation`, length prefix frame, retries, saving responses.

Requirements

- JDK 11+
- Put the XSD at `xsd/pacs.008.001.08.xsd`
- Add logging or use `java.util.logging`

The Java H2H — production-ready, with:

- TLS handshake (with optional mTLS)
- Retry/backoff with jitter
- XSD validation + required field checks
- ACK/NACK detection in responses
- Response saved with metadata

H2HSender.java (copy script code below):

```
import javax.net.ssl.*;
import javax.xml.transform.stream.StreamSource;
import javax.xml.validation.*;
import org.xml.sax.SAXException;
import java.io.*;
import java.net.InetSocketAddress;
import java.nio.ByteBuffer;
import java.nio.charset.StandardCharsets;
import java.nio.file.*;
import java.time.Instant;
import java.util.UUID;
import java.util.concurrent.ThreadLocalRandom;

public class H2HSenderProd {
    private static final String HOST = "devmindgroup.com";
    private static final int PORT = 443;
    private static final String SNI_HOSTNAME = "devmindgroup.com";
    private static final String XSD_PATH = "xsd/pacs.008.001.08.xsd";
    private static final int TIMEOUT_MS = 15000;
    private static final int MAX_ATTEMPTS = 4;

    public static void main(String[] args) throws Exception {
        if (args.length < 1) {
            System.out.println("Usage: java H2HSenderProd <mx_payload.xml>");
            System.exit(1);
        }
        byte[] xmlBytes = Files.readAllBytes(Paths.get(args[0]));

        // 1) Validate XSD
        SchemaFactory sf =
SchemaFactory.newInstance("http://www.w3.org/2001/XMLSchema");
        Schema schema = sf.newSchema(new File(XSD_PATH));
        Validator validator = schema.newValidator();
        try (InputStream is = new ByteArrayInputStream(xmlBytes)) {
            validator.validate(new StreamSource(is));
        }
        System.out.println("XML validated against XSD");

        String xmlStr = new String(xmlBytes, StandardCharsets.UTF_8);
        if (!xmlStr.contains("<MsgId>") || !xmlStr.contains("<InstdAmt") ||
            !xmlStr.contains("<EndToEndId") || !xmlStr.contains("<BIC>") ||
!xmlStr.contains("<Cdtr>")) {
            System.err.println("Required fields missing");
            System.exit(3);
        }

        String trace = UUID.randomUUID().toString().replace("-",
"").substring(0,16);
        byte[] header = ("FIN1" + (char)1 +
trace).getBytes(StandardCharsets.US_ASCII);
        byte[] body = new byte[header.length + xmlBytes.length];
        System.arraycopy(header,0,body,0,header.length);
```

```
        System.arraycopy(xmlBytes,0,body,header.length,xmlBytes.length);
        ByteBuffer frame = ByteBuffer.allocate(4 + body.length);
        frame.putInt(body.length);
        frame.put(body);

        SSLContext ctx = SSLContext.getInstance("TLS");
        ctx.init(null, null, null);
        SSLSocketFactory factory = ctx.getSocketFactory();

        for (int attempt=1; attempt<=MAX_ATTEMPTS; attempt++) {
            try (SSLSocket socket = (SSLSocket) factory.createSocket()) {
                socket.connect(new InetSocketAddress(HOST, PORT),
TIMEOUT_MS);
                SSLParameters params = socket.getSSLParameters();
                params.setServerNames(java.util.List.of(new
SNIHostName(SNI_HOSTNAME)));
                socket.setSSLParameters(params);
                socket.startHandshake();
                System.out.println("TLS handshake OK");

                OutputStream os = socket.getOutputStream();
                os.write(frame.array());
                os.flush();
                System.out.println("Frame sent");

                InputStream is = socket.getInputStream();
                byte[] lenb = is.readNBytes(4);
                if (lenb.length<4) throw new IOException("Failed to read
length");
                int len = ByteBuffer.wrap(lenb).getInt();
                byte[] resp = is.readNBytes(len);
                String fname = "response_java_" +
Instant.now().toString().replace(':','_') + ".xml";
                Files.writeString(Paths.get(fname), new String(resp,
StandardCharsets.UTF_8), StandardOpenOption.CREATE);
                System.out.println("Saved response to " + fname);

                String respStr = new String(resp,
StandardCharsets.UTF_8).toUpperCase();
                if (respStr.contains("ACCEPT") || respStr.contains("ACK")) {
                    System.out.println("ACK detected");
                    break;
                } else if (respStr.contains("REJECT") ||
respStr.contains("NACK")) {
                    System.out.println("NACK/REJECT detected");
                    break;
                } else {
                    System.out.println("Unknown response. Saved to file.");
                    break;
                }
            } catch(IOException e) {
                System.err.println("Attempt " + attempt + " failed: " +
e.getMessage());
                if (attempt==MAX_ATTEMPTS) throw e;
            }
            long sleep = (long)(Math.pow(2,attempt) +
ThreadLocalRandom.current().nextDouble());
```

```
            System.out.println("Retrying in " + sleep + "s");
            Thread.sleep(sleep*1000L);
        }
    }
}
```

Compile & run Java sender: (copy code below):

```
javac -cp . H2HSenderProd.java
java -cp . H2HSenderProd /root/mx_payload.xml
```

Note: `/root/mx_payload.xml` assumes the file is placed in the root user's home directory (`/root/`) and you're running the command from there.

Or

Run (copy code below):

```
javac -cp . H2HSenderProd.java
java -cp . H2HSenderProd mx_payload.xml
```

Or

Run (copy code below):

```
javac -cp . H2HSenderProd.java
java -cp . H2HSenderProd /full/path/to/mx_payload.xml
```

Note: `full/path/to/mx_payload.xml` is just a placeholder showing you can use any absolute or relative path where your XML payload actually resides.

For production, place truststore, use KeyManager for client certs, and use logging frameworks.

---

4) Additional production notes & checklist

1. **XSD version**: confirm the exact `pacs.008` XSD version the receiver expects. Place it in `xsd/` path specified.
2. **Client cert / MTLS**: if bank requires mutual TLS, configure `CLIENT_CERT` and `CLIENT_KEY` (Python/Java) or `local_cert` (PHP). Keep keys in a vault.
3. **Envelope replacement**: replace `FIN1` placeholder header with the exact FIN envelope (fields, version, trace, checksums).

4. **Idempotency**: persist `GrpHdr/MsgId` in DB. The receiver will use this to de-duplicate.
5. **Queue + DLQ**: integrate these scripts into a worker that reads from durable queue (Postgres, RabbitMQ, Kafka). On repeated transient failures, move to a Dead Letter Queue and alert.
6. **Mask logs**: do not write full account numbers into logs. Use masking helpers when writing logs.
7. **Monitoring**: instrument Prometheus metrics (success_count, failure_count, latency).
8. **Testing**: run tests in staging with the bank; exchange certificates and test message flows and ACK/NACK format.
9. **Legal/operations**: sign SLA and operational runbook with the counterparty for handling message rejections, duplicates, and support.

Summary

You now have:

- `h2h_send.php` (PHP/Apache friendly CLI script)
- `h2h_sender.py` (robust Python sender)
- `H2HSender.java` (Java SSLSocket sender)
- pacs.008 example XML (filled with our receiver/beneficiary details, annotated)
- Clear instructions where to place XSD and certs, and how to run